Exceptions

15-3-26

Aim – Demonstrate exceptions in python.

Theory – Exception handling in Python is used to manage runtime errors without stopping program execution. The try block contains code that may cause errors, while the except block handles specific exceptions like ZeroDivisionError, ValueError, IndexError, KeyError, and FileNotFoundError. Custom exceptions allow programmers to define and handle application-specific errors.

A1.

```
# 1. ZeroDivisionError
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("ZeroDivisionError:", e)


# 2. ValueError
try:
    num = int("abc")
except ValueError as e:
    print("ValueError:", e)


# 3. IndexError
try:
    lst = [1, 2, 3]
    print(lst[5])
except IndexError as e:
    print("IndexError:", e)


# 4. KeyError
try:
    d = {'a': 1}
    print(d['b'])
except KeyError as e:
    print("KeyError:", e)
```

```
# 5. FileNotFoundError
try:
    with open("nonexistentfile.txt", "r") as f:
        content = f.read()
except FileNotFoundError as e:
    print("FileNotFoundError:", e)


# Custom Exception
class MyCustomException(Exception):
    pass


try:
    raise MyCustomException("This is a custom exception!")
except MyCustomException as e:
    print("MyCustomException:", e)
```

```
PS C:\Users\tarun\Downloads\TSEC-SEM2-PYTHON> & c:\python314\python.exe "c:/Users/t
ignment 6/A1.py"
ZeroDivisionError: division by zero
ValueError: invalid literal for int() with base 10: 'abc'
IndexError: list index out of range
KeyError: 'b'
FileNotFoundError: [Errno 2] No such file or directory: 'nonexistentfile.txt'
MyCustomException: This is a custom exception!
```

Conclusion

Exception handling improves program reliability by preventing crashes and allowing developers to manage different runtime errors in a controlled way.